UE24CS341A — SOFTWARE ENGINEERING

SOFTWARE ARCHITECTURE & DESIGN SPECIFICATION

Jackfruit File Compression Tool

Layered Architecture, UML Design Models & STRIDE Threat Model

Version 0.1.0 | DRAFT | September 2026

Project	Jackfruit — Basic ZIP-Style File Compression Tool (Huffman Coding)
Team	Saatwik • Tanmayi • Yuvraj • Vibhav
Lead Author	Saatwik (Architecture Owner & Integrator)
Course	UE24CS341A — Software Engineering, PES University
Document Control	SAD-JACK-0.1.0 • Pending Team Sign-off

Revision History & Approvals

Revision History

Version	Date	Author	Change Summary	Approval
0.1.0	28-09-2026	Saatwik	Initial baseline mapped to Course SAD Template (Layered model, STRIDE threat model, 2 sequence diagrams, API specs)	Pending Team Sign-off

Approvals

Role	Name	Signature / Email	Date
Architecture Lead & Integrator	Saatwik	saatwik455@gmail.com	28-09-2026
Testing & QA Lead	Tanmayi	tanmayi.qa@jackfruit.local	11-10-2026
CI/CD Lead	Yuvraj	yuvraj.devops@jackfruit.local	11-10-2026
Security & Quality Lead	Vibhav	vibhav.sec@jackfruit.local	11-10-2026
Course Coordinator / Evaluator	Instructor	Faculty Review	Pending

Table of Contents

1. Introduction 4

2. Document Overview 5

3. Architecture 6

- 3.1–3.2 Goals, Constraints, Stakeholders 6
- 3.3–3.4 Component Diagrams & Descriptions 6
- 3.3.1 UML Component Diagram (Subsystem View) 6
- 3.5–3.6 Architecture Pattern & Technology Stack 8
- 3.7–3.8 Risks & Traceability 9
- 3.9 Security Architecture (STRIDE) 9

4. Design 10

- 4.1–4.2 Design Overview & Sequence Diagrams 10
- 4.3 API Design 13
- 4.4–4.6 Error Handling, UX, Open Issues 15

5. Appendices 18

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design for the Jackfruit Compression System (Release 1.0). It defines module decomposition, chosen architectural patterns, structural and behavioural UML models, public API signatures, and a formal security threat model.

1.2 Scope

The design encompasses all internal components responsible for single-file Huffman compression, decompression, archive serialization (`.jack` format), CRC-32 integrity validation, and the menu-driven interactive terminal interface. It deliberately stops at the boundary of the operating system's filesystem API — Jackfruit does not design its own storage layer, journaling, or atomic-write mechanism, relying instead on the C++ standard library's `<fstream>` and `<filesystem>` facilities, since reinventing filesystem semantics is explicitly out of scope for a course project of this size.

This document is a living artefact: Section 4.6 (Open Issues & Next Steps) is updated at every sprint boundary, and any structural change to a public interface described in Section 4.3 is expected to be reflected here in the same pull request that changes the code, not after the fact.

1.3 Audience

- **Developers** (Saatwik, Tanmayi, Yuvraj, Vibhav) — for module implementation and integration against public interfaces.
- **QA Lead** (Tanmayi) — for constructing component mockups and integration tests.
- **Security Lead** (Vibhav) — for verifying STRIDE mitigations and static analysis quality gates.
- **Course Instructors & Evaluators** — for evaluating adherence to object-oriented and modular design principles.

1.4 Definitions and Acronyms

Term	Definition
ADR	Architecture Decision Record.
API	Application Programming Interface.
CRC-32	32-bit Cyclic Redundancy Check (IEEE 802.3).
MSB	Most Significant Bit.
SAD	Software Architecture and Design Specification.
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege.
UML	Unified Modeling Language.

2. Document Overview

2.1 How to Use This Document

Developers should refer to Section 3 for high-level module decomposition, dependency rules, and threat modelling, and Section 4 for sequence flows, class models, and C++ header signatures.

2.2 Related Documents

- Software Requirements Specification — `docs/srs/srs-v0.1.md`
- Binary Archive Format Specification — `docs/design/archive-format.md`
- Software Test Plan — `docs/validation/`
- Requirements Traceability Matrix — `docs/srs/srs-v0.1.md#8-requirements-traceability-matrix-rtm`

3. Architecture

3.1 Goals & Constraints

Primary Goals: High cohesion, low coupling, deterministic bitstream execution, cross-platform portability, streaming memory footprint (≤64 MB).

Constraints: Pure standard C++17, zero third-party compression libraries, terminal menu-driven interface, Big-Endian binary container standard.

These constraints are not arbitrary: the zero-third-party-library rule exists so that the team implements and can explain every byte of the compression pipeline itself (a direct requirement of the course's "must understand what you submitted" evaluation criterion), the Big-Endian standard exists so that an archive produced on one teammate's machine is guaranteed byte-identical when parsed on another's regardless of host CPU architecture, and the streaming memory ceiling exists so that the tool remains usable on the modest hardware typical of a student laptop rather than assuming server-class memory availability.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
End Users	Fast execution, high compression ratio, zero data loss, clear terminal progress feedback.
Academic Evaluators	Modularity, unit test coverage (≥80%), automated CI/CD pipeline, and design traceability.
Maintenance Team	Minimal inter-module coupling with stable interfaces under `include/`.

3.3 Component (UML) Diagram

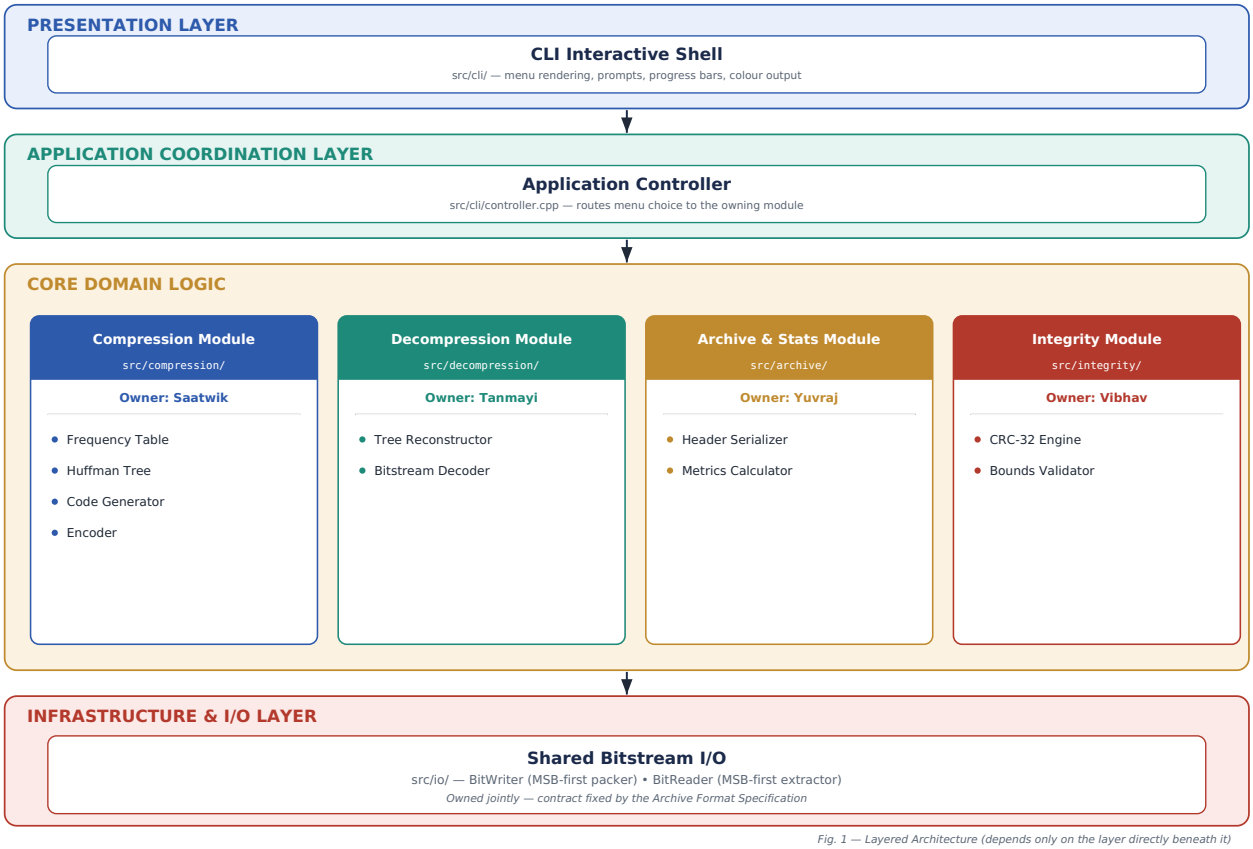


Fig. 1 — Layered Architecture (depends only on the layer directly beneath it)

3.3.1 UML Component Diagram — Subsystem View

Figure 1 shows *what depends on what* layer by layer. Figure 2 below re-expresses the same architecture as a standard UML 2 component diagram, showing *how the modules actually connect*: the three subsystems (Presentation &

Coordination, Core Domain Logic, Infrastructure & I/O), the ports each component exposes, the interfaces that bind components inside a subsystem, and the usage dependencies that cross subsystem boundaries. Every request that enters from the user is delegated through a subsystem port to exactly one owning module, and every module reaches the file system only through the Infrastructure subsystem — the same one-directional dependency rule stated in Section 3.5, now visible at the port level.

Jackfruit — UML Component Diagram (Subsystem View)

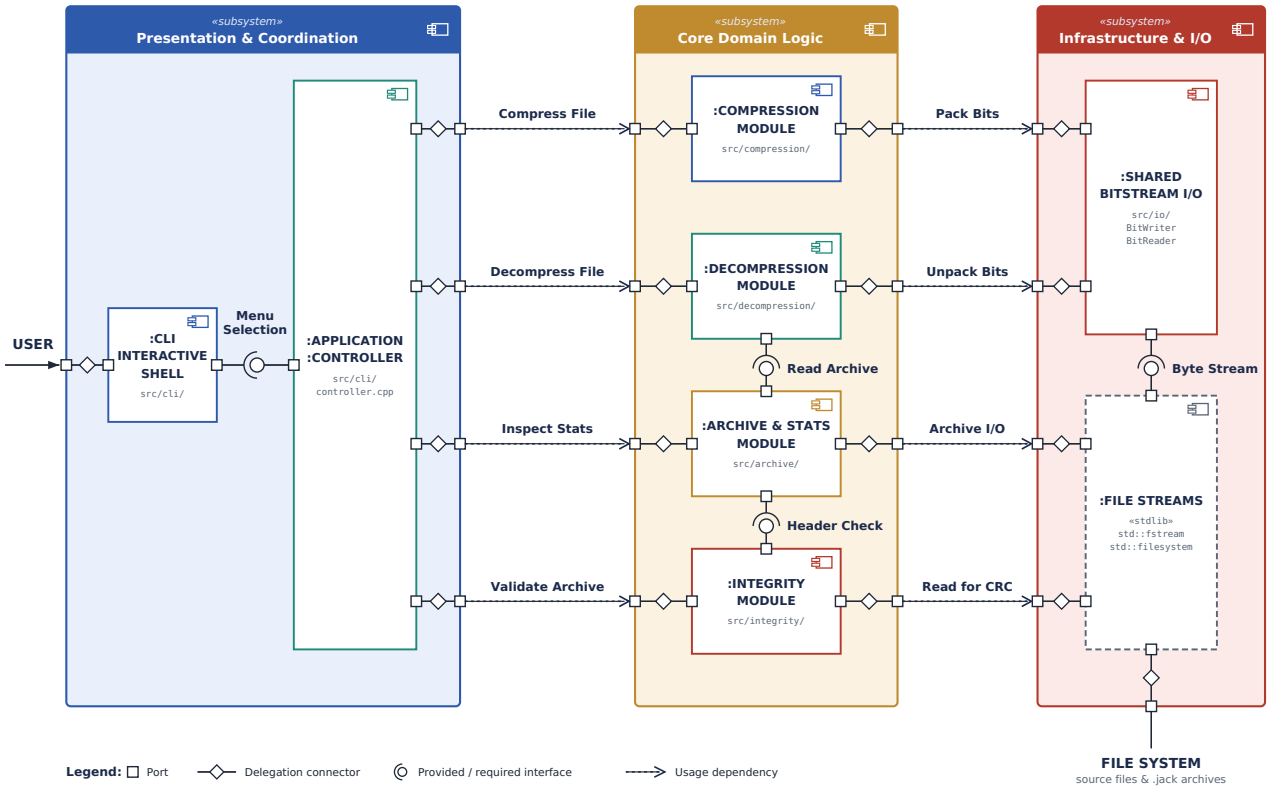


Fig. 2 — UML Component Diagram: subsystems, ports, interfaces and usage dependencies

INTERFACE & DEPENDENCY SUMMARY

Connector	Kind	From → To	Purpose / Requirements Served
Menu Selection	Interface	CLI Interactive Shell ↔ Application Controller	Passes the user's menu choice from shell to controller (JACK-F-011, JACK-F-016, JACK-NF-005).
Compress File	Dependency	Application Controller → Compression Module	Menu option 1 — start UC-01 (JACK-F-001 to JACK-F-004).
Decompress File	Dependency	Application Controller → Decompression Module	Menu option 2 — start UC-02 (JACK-F-005 to JACK-F-008).
Inspect Stats	Dependency	Application Controller → Archive & Stats Module	Menu option 3 — header-only statistics (JACK-F-009 to JACK-F-012).
Validate Archive	Dependency	Application Controller → Integrity Module	Menu option 4 — structural and CRC-32 validation (JACK-F-013 to JACK-F-016).
Read Archive	Interface	Decompression Module ↔ Archive & Stats Module	Parsed header and frequency table via `ArchiveReader` (JACK-F-005, JACK-F-006).
Header Check	Interface	Archive & Stats Module ↔ Integrity Module	Magic, version and bounds checks on a parsed header (JACK-F-013, JACK-F-014, JACK-SR-001 to 003).
Pack Bits	Dependency	Compression Module → Shared Bitstream I/O	MSB-first encoding through `BitWriter` (JACK-F-003).
Unpack Bits	Dependency	Decompression Module → Shared Bitstream I/O	MSB-first decoding through `BitReader` (JACK-F-007).
Archive I/O	Dependency	Archive & Stats Module → File Streams	Reading and writing the `.jack` container (JACK-F-004, JACK-F-009).
Read for CRC	Dependency	Integrity Module → File Streams	Reads file bytes to compute and verify CRC-32 (JACK-F-015, JACK-SR-004).
Byte Stream	Interface	Shared Bitstream I/O ↔ File Streams	`std::istream` / `std::ostream` used by the bit packer (JACK-NF-002).

Notes: (1) *File Streams* models the C++ standard-library dependency in SRS Section 3.3 (`<fstream>`, `<filesystem>`); it is external, not project code, so it is drawn dashed. (2) One representative interface is shown per module pair; the finer-grained calls within a single operation are shown step by step in the sequence diagrams of Section 4.2.

3.4 Component Descriptions

1. **CLI Shell** (`src/cli/`) — manages user interaction, menu rendering, input prompt validation, real-time progress indicators, and terminal colour highlighting.
2. **Compression Module** (`src/compression/` — Saatwik) — computes byte frequency distributions, constructs the Huffman binary tree using a priority min-heap, generates canonical variable-length prefix codes, and converts byte buffers to bitstreams.
3. **Decompression Module** (`src/decompression/` — Tanmayi) — parses tree frequency metadata, reconstructs the identical Huffman prefix tree, decodes packed bitstreams symbol-by-symbol, and writes original byte streams.
4. **Archive & Statistics Module** (`src/archive/` — Yuvraj) — handles binary reading and writing of the 18-byte fixed header and metadata tables in Big-Endian order. Computes file size ratios and percentage space savings.
5. **Integrity Module** (`src/integrity/` — Vibhav) — computes IEEE 802.3 CRC-32 checksums and performs structural validation (magic signature, supported version, payload boundary consistency).
6. **Shared Bitstream I/O** (`src/io/` — Shared) — implements `BitWriter` (MSB-first accumulator with zero-padding flush) and `BitReader` (MSB-first extractor stopping strictly at `total_bits`).

3.4.1 Design Principles Applied

- **Single Responsibility Principle** — each class in the table above owns exactly one concern (e.g. `FrequencyTable` only counts bytes; it never touches I/O or the archive format).
- **RAII Resource Management** — file handles and dynamically allocated Huffman tree nodes are owned by stack-lifetime objects or smart pointers, so no module performs manual `new` / `delete` pairing.
- **Dependency Direction Discipline** — a module may only include headers from its own layer or the layer directly beneath it in Figure 1; `src/compression/` is not permitted to include anything from `src/cli/` , which is what makes each module independently unit-testable without constructing the whole application.
- **Interface Segregation at Module Boundaries** — cross-module calls go through the narrow public headers in `include/` (Section 4.3), never through a module's internal implementation files, so an internal refactor inside one module cannot silently break another.

3.4.2 Class-Level Design Notes

The following classes are the primary units of implementation inside each module; their full interfaces are formalised in Section 4.3 for the two components the course requires in detail, and summarised here for completeness across the whole system.

Module	Key Class(es)	Responsibility
src/compression/	`FrequencyTable` , `HuffmanTree` , `Compressor`	Count byte frequencies; build a deterministic canonical prefix tree via a priority min-heap; orchestrate end-to-end compression and delegate to `ArchiveWriter` and `BitWriter` .
src/decompression/	`TreeReconstructor` , `Decompressor`	Rebuild the identical Huffman tree from stored frequency metadata; drive symbol-by-symbol decoding via `BitReader` and verify the result via `IntegrityEngine` .
src/archive/	`ArchiveHeader` , `ArchiveWriter` , `ArchiveReader`	Define the in-memory header representation; serialize/deserialize it and the frequency table to/from the exact byte layout in the Archive Format Specification; compute statistics from a parsed header.
src/integrity/	`IntegrityEngine` , `BoundsValidator`	Compute and verify IEEE 802.3 CRC-32 checksums; enforce magic-byte, version, and structural-bound checks before any payload is trusted.
src/io/	`BitWriter` , `BitReader`	Pack/unpack individual bits into/from a byte stream under the shared MSB-first contract (see Archive Format Specification §4).
src/cli/	`MenuShell` , `ApplicationController`	Render the interactive menu, collect and validate user input, and route the chosen operation to the owning module's public entry point.

3.5 Chosen Architecture Pattern and Rationale

The system adopts a **Layered Architecture Pattern**.

Why Layered Architecture? Layering provides strict separation between user presentation (`src/cli`), domain algorithms (`src/compression` , `src/decompression`), and raw I/O (`src/io`). Each layer only depends on the layer directly beneath it, facilitating independent unit testing and mock injection.

Alternatives Rejected:

- **Monolithic Script Style** — rejected because it violates course modularity requirements and prevents parallel team development.

- **Pipes-and-Filters / Streaming Shell Pipeline** — rejected because interactive menu navigation and multi-stage header-payload writing require random file seeking for header updates (such as writing `total_bits` after payload completion).

3.6 Technology Stack & Data Stores

- **Programming Language:** Standard C++17 (compiled with Clang++ or GCC).
- **Build System:** CMake (≥3.14).
- **Test Framework:** GoogleTest (v1.14.0 via CMake `FetchContent`).
- **Data Store / File Storage:** Binary `.jack` container files adhering to the byte layout in `docs/design/archive-format.md`.

3.7 Risks & Mitigations

ID	Description	Impact	Mitigation Strategy
RISK-01	High memory usage on large file compression	High	Stream-based chunk processing using fixed 64 KB buffers rather than loading entire files into memory.
RISK-02	Bitstream decoding overshoot from trailing byte padding	High	Store exact 64-bit `total_bits` in the header; `BitReader` halts reading as soon as `bits_read = total_bits`.
RISK-03	Endianness mismatch across Intel x86 and Apple Silicon ARM CPUs	Medium	All multi-byte header integers are serialized and deserialized in explicit Big-Endian (Network Byte Order).
RISK-04	Single-symbol or empty file crash in Huffman tree builder	Medium	Special-case flags: 0-byte files generate empty archives (`IS_EMPTY=1`); single-symbol files assign a trivial 1-bit code `0`.

3.8 Traceability to Requirements

Module	Requirements Addressed
src/cli/	JACK-F-001, JACK-F-011, JACK-F-016, JACK-NF-005, JACK-SR-005
src/compression/	JACK-F-001, JACK-F-002, JACK-F-003, JACK-F-004, JACK-NF-001
src/decompression/	JACK-F-005, JACK-F-006, JACK-F-007, JACK-F-008, JACK-NF-001
src/archive/	JACK-F-004, JACK-F-005, JACK-F-009, JACK-F-010, JACK-F-012, JACK-NF-004
src/integrity/	JACK-F-013, JACK-F-014, JACK-F-015, JACK-F-016, JACK-SR-001 to 004
src/io/	JACK-F-003, JACK-F-007, JACK-NF-002, JACK-SR-003

3.9 Security Architecture (STRIDE Threat Model)

> **INSTRUCTOR RULE**
> STRIDE Threat Model required in Section 3.9.

Category	Specific Threat in Jackfruit	Sev.	Architectural Mitigation
Spoofing	Attacker provides an arbitrary binary file renamed to `.jack` to trick the decompressor.	Medium	**Magic Byte Gating:** `ArchiveReader` checks for ASCII "JACK" (`0x4A414B43`) and version 1 before allocating memory (JACK-SR-001).
Tampering	Attacker modifies bits in compressed payload or frequency table.	High	**CRC-32 Verification:** full IEEE 802.3 CRC-32 checksum of original file is verified after decompression; fails with alert on mismatch (JACK-SR-004).
Repudiation	Ambiguous failure without diagnostic cause during corrupt archive processing.	Low	**Structured Error Codes:** explicit enum error reporting (`INVALID_MAGIC`, `UNSUPPORTED_VERSION`, `CHECKSUM_MISMATCH`, `TRUNCATED_STREAM`).
Information Disclosure	Sensitive data or buffer leftovers leaked into error logs or uninitialized memory.	Low	**Zero Memory Leakage:** internal buffers zero-initialized; no memory contents dumped to terminal logs.
Denial of Service	Crafted header with `symbol_count > 256` or enormous size to exhaust heap memory.	High	**Strict Bound Verification:** header parser enforces 0≤N≤256 and limits payload bit count to valid file length (JACK-SR-002, JACK-SR-003).
Elevation of Privilege	Path traversal attack via output file path (e.g. `./../etc/passwd`).	High	**Path Sanitization:** file path resolver strips directory escape sequences before file creation (JACK-SR-005).

4. Design

4.1 Design Overview

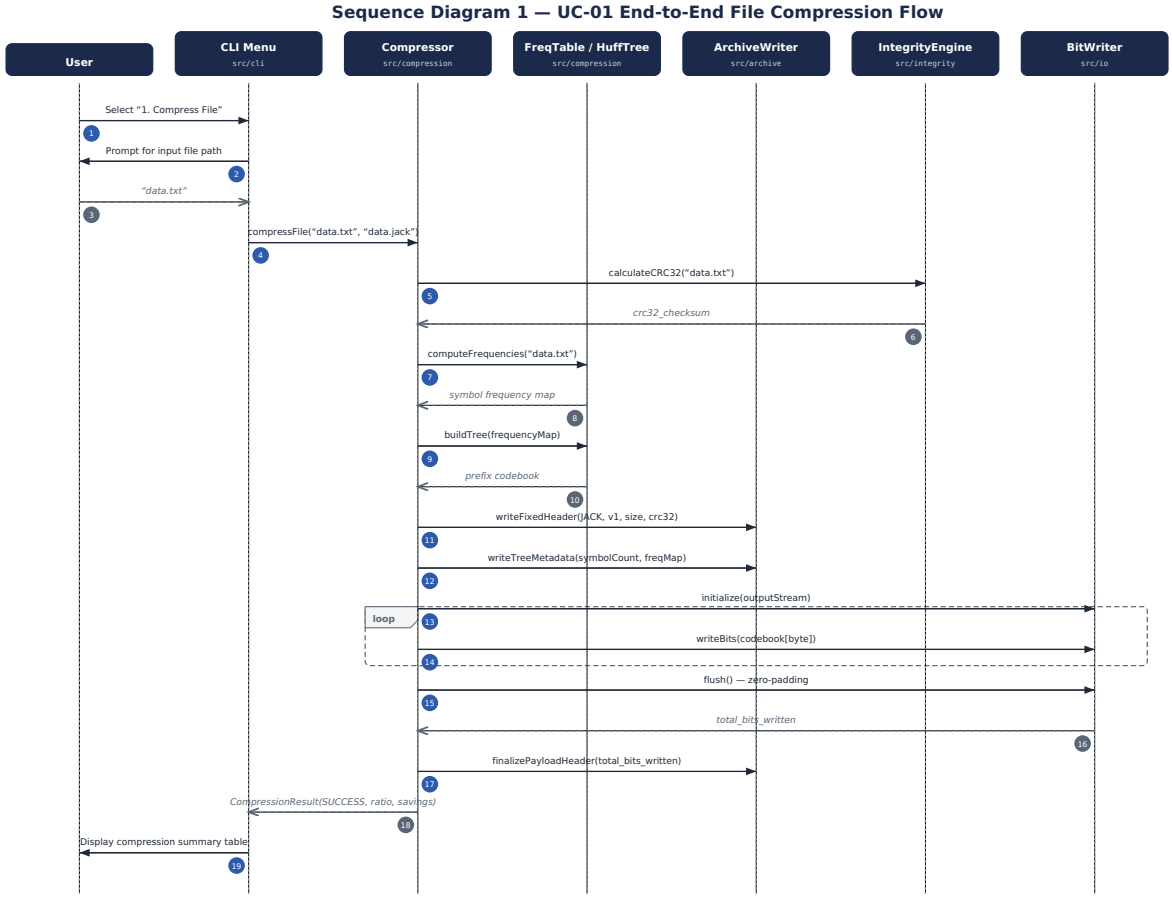
The system follows standard object-oriented C++17 design principles. Classes are encapsulated, dynamic memory in Huffman trees is managed using smart pointers or RAII destructors, and bitstreams are abstracted via stream wrappers. Every cross-module operation returns a structured result object (an enum status plus relevant data) rather than throwing exceptions across module boundaries, which keeps error handling explicit and keeps the control-flow graph of each function easy to reason about during code review — a deliberate trade-off of some C++ idiom in favour of reviewability, since every line of this codebase must be explainable live at the final demo.

The two sequence diagrams below were chosen specifically because they are the two flows that cross every architectural layer: compression exercises the full write path (CLI → Compressor → FrequencyTable/HuffmanTree → ArchiveWriter → IntegrityEngine → BitWriter), and decompression exercises the full read-and-verify path in the opposite direction, including the one behavioural branch (checksum match vs. mismatch) that the course's "at least 2 flows" requirement is best demonstrated by.

4.2 UML Sequence Diagrams

> **INSTRUCTOR RULE**
> At least 2 sequence diagrams covering 2 specific project flows.

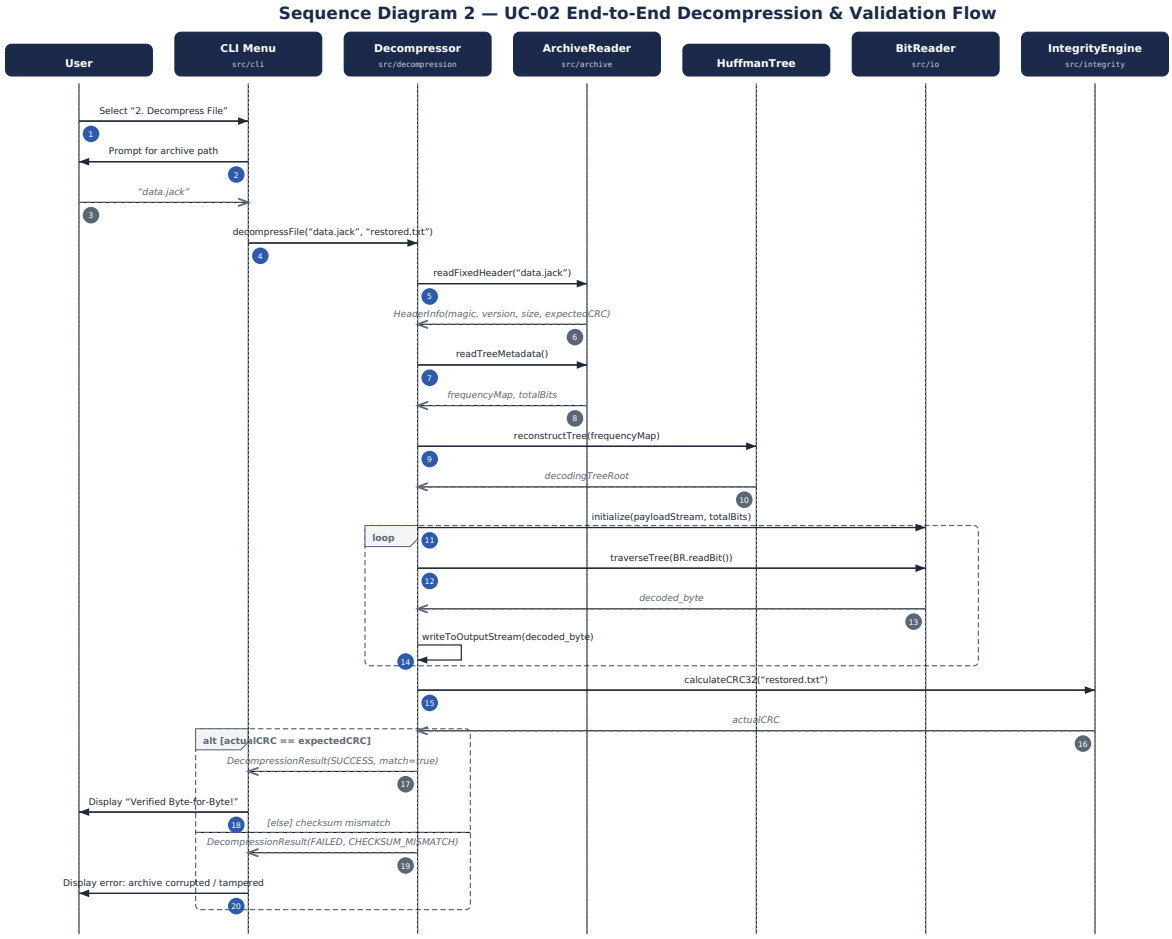
Sequence Diagram 1: UC-01 End-to-End File Compression Flow



Three details in this flow are easy to miss on a first reading but matter for correctness. First, the CRC-32 of the *original* file (steps 5–6) is computed and stored before any encoding happens, so the checksum that decompression later verifies against is independent of any bug in the encoder itself. Second, the header is written in two passes:

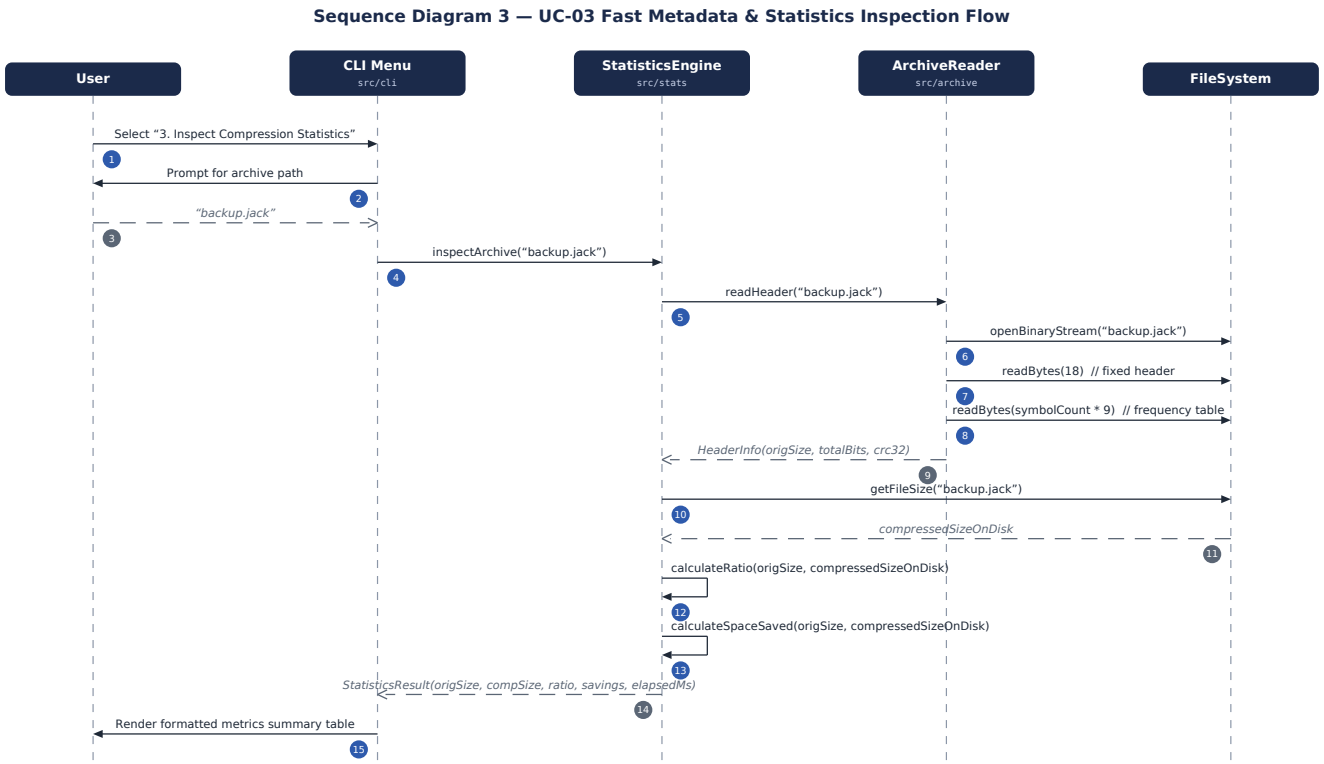
`writeFixedHeader` / `writeTreeMetadata` (steps 11–12) happen before the bitstream is known, and `finalizePayloadHeader` (step 17) seeks back to patch in `total_bits_written` only once `flush()` has returned it — this two-pass write is exactly why a pure streaming Pipes-and-Filters architecture was rejected in Section 3.5. Third, the loop over "each byte in data.txt" (step 14) is where the ≥5 MB/s throughput requirement (JACK-NF-001) is actually spent, making it the natural place to look first if a future profiling pass finds the pipeline too slow.

Sequence Diagram 2: UC-02 End-to-End Decompression & Validation Flow

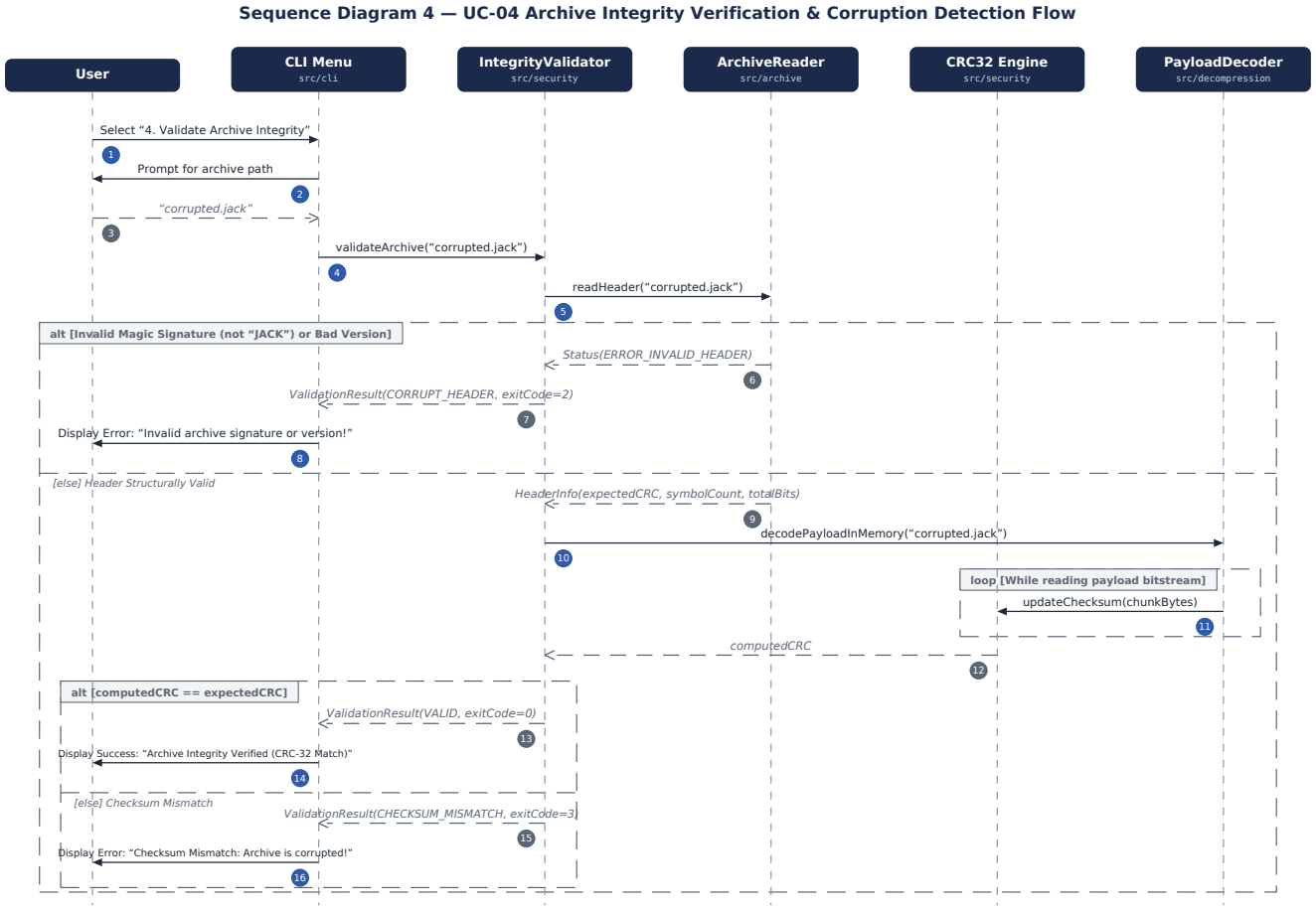


The `alt` block at the end of this flow is the single most safety-critical branch point in the whole system: every decompression, successful or not, ends by comparing a freshly computed CRC-32 of the restored bytes against the CRC-32 stored at compression time, and the user-visible outcome is driven entirely by that comparison rather than by whether the decode loop happened to terminate without crashing. This is a deliberate defence against a subtle class of bug where a decoder silently produces *plausible but wrong* output (for example, from a tree-reconstruction error) — without the final CRC gate, such a bug could pass casual visual inspection of a text file while still being wrong at the byte level.

Sequence Diagram 3: UC-03 Fast Metadata & Statistics Inspection Flow (Owner: Yuvraj)



Sequence Diagram 4: UC-04 Archive Integrity Verification & Corruption Detection Flow (Owner: Vibhav)



4.3 API Design (Interface Definitions)

> **INSTRUCTOR RULE**
> Formal interface definitions for at least 2 components.

The two components formalised below were selected because they sit at the two ends of the data pipeline — the highest-level orchestration entry point a CLI caller sees, and the lowest-level bit-packing primitive every other module ultimately depends on — and because together they illustrate both a result-object-returning interface style (Compressor) and a stateful streaming-object interface style (BitWriter/BitReader). A third component, Archive Serialization, is included for completeness since it is the interface the Archive Format Specification is written against.

Component 1: Compression Subsystem (include/compression/compressor.hpp)

This is the public entry point the CLI shell calls for menu option 1. Its signature intentionally exposes only file paths in and a result object out — none of `FrequencyTable`, `HuffmanTree`, or `BitWriter` appear in this header, which is what lets the internal algorithm be refactored freely without breaking the CLI's compile-time contract.

```
#pragma once
#include <string>
#include <cstdint>
#include <vector>

enum class CompressionStatus {
    SUCCESS,
    ERROR_FILE_NOT_FOUND,
    ERROR_INPUT_EMPTY,
    ERROR_CANNOT_WRITE_ARCHIVE,
    ERROR_UNKNOWN
};

struct CompressionResult {
    CompressionStatus status;
    std::string errorMessage;
    uint64_t originalBytes;
    uint64_t compressedBytes;
    double compressionRatio;
    double percentageSaved;
    double executionTimeMs;
};

class Compressor {
public:
    Compressor() = default;
    ~Compressor() = default;

    // Compresses input file into a .jack archive
    CompressionResult compressFile(const std::string& inputPath,
                                   const std::string& outputPath);
};
```

Component 2: Bitstream Packing Engine (include/io/bit_stream.hpp)

This is the lowest layer in Figure 1 and the one component both Saatwik's encoder and Tanmayi's decoder depend on identically. `BitWriter` and `BitReader` are intentionally stateful, constructed once around a stream and driven bit-by-bit or sequence-by-sequence, rather than exposing free functions — this lets the internal partial-byte buffer (`buffer_` / `bitCount_`) stay encapsulated instead of being threaded through every call site.

```
#pragma once
#include <iostream>
#include <cstdint>
#include <string>

class BitWriter {
public:
    explicit BitWriter(std::ostream& outputStream);
    ~BitWriter();

    void writeBit(uint8_t bit);                     // MSB-first
    void writeBits(const std::string& bitSequence);  // '0'/'1' chars
    void flush();                                    // zero-pad tail
    uint64_t getTotalBitsWritten() const;

private:
    std::ostream& out_;
    uint8_t  buffer_{0};
    uint8_t  bitCount_{0};
    uint64_t totalBits_{0};
};

class BitReader {
public:
    explicit BitReader(std::istream& inputStream, uint64_t totalBits);
    ~BitReader() = default;

    bool readBit(uint8_t& bit);   // false once totalBits reached
    bool hasMoreBits() const;

private:
    std::istream& in_;
    uint64_t totalBits_;
    uint64_t bitsRead_{0};
    uint8_t  buffer_{0};
    uint8_t  bitIndex_{8};
};
```

Component 3: Archive Serialization (include/archive/archive.hpp)

These two classes are static-method utilities rather than stateful objects, reflecting that reading or writing an archive header is a single atomic operation with no meaningful partial state between calls. `ArchiveHeader` is the in-memory mirror of the byte layout defined field-for-field in the Archive Format Specification; any change to one must be made to the other in the same commit.

```
#pragma once
#include <string>
#include <vector>
#include <cstdint>

struct ArchiveHeader {
    char     magic[4];        // "JACK"
    uint8_t  version;         // 1
    uint8_t  flags;           // bitwise flags
    uint64_t originalSize;    // original uncompressed size
    uint32_t checksumCRC32;   // CRC-32 checksum
    uint16_t symbolCount;     // unique symbol count (0..256)
    uint64_t totalBits;       // total valid encoded payload bits
};

class ArchiveWriter {
public:
    static bool writeArchive(const std::string& outputPath,
                             const ArchiveHeader& header,
                             const std::vector<std::pair<uint8_t,uint64_t>>& frequencies,
                             const std::vector<uint8_t>& payloadBytes);
};

class ArchiveReader {
public:
    static bool readHeader(const std::string& archivePath,
                           ArchiveHeader& header,
                           std::vector<std::pair<uint8_t,uint64_t>>& frequencies);
};
```

Component 4: Statistics Engine (include/stats/statistics_engine.hpp — Owner: Yuvraj)

```
#pragma once
#include <string>
#include <cstdint>

struct StatisticsResult {
    bool success;
    std::string archivePath;
    uint64_t originalSizeBytes;
    uint64_t compressedSizeBytes;
    double compressionRatio;
    double percentageSaved;
    uint32_t checksumCRC32;
    uint16_t uniqueSymbolCount;
    double inspectionDurationMs;
    std::string errorMessage;
};

class StatisticsEngine {
public:
    StatisticsEngine() = default;
    ~StatisticsEngine() = default;

    // Fast header inspection without full decompression
    StatisticsResult inspectArchive(const std::string& archivePath);

    // Formatted terminal summary rendering
    std::string formatSummaryTable(const StatisticsResult& stats) const;
};
```

Component 5: Integrity Engine & CRC-32 (include/security/integrity_validator.hpp & include/security/crc32.hpp — Owner: Vibhav)

```
#pragma once
#include <string>
#include <cstdint>
#include <vector>

enum class ValidationStatus {
    VALID = 0,
    ERROR_FILE_NOT_FOUND = 1,
    ERROR_INVALID_HEADER = 2,
    ERROR_CHECKSUM_MISMATCH = 3,
    ERROR_TRUNCATED_PAYLOAD = 4
};

struct ValidationResult {
    ValidationStatus status;
    int exitCode;
    uint32_t expectedChecksum;
    uint32_t computedChecksum;
    std::string message;
};

class CRC32 {
public:
    static uint32_t calculate(const uint8_t* data, size_t length);
    static uint32_t update(uint32_t currentCRC, const uint8_t* data, size_t length);
    static uint32_t calculateFile(const std::string& filePath);
};

class IntegrityValidator {
public:
    IntegrityValidator() = default;
    ~IntegrityValidator() = default;

    // Two-stage structural and cryptographic validation
    ValidationResult validateArchive(const std::string& archivePath);
};
```

4.4 Error Handling, Logging & Monitoring

- **Error Codes:** no unhandled C++ exceptions leak across module boundaries. Operations return structured result

objects with explicit status enums.

- **Console Feedback:** standard output is reserved for user prompts; error descriptions are printed in colour to `std::cerr`.
- **Zero Sensitive Data in Logs:** no confidential file paths or raw byte dumps are emitted.

The table below consolidates the structured error enumerations referenced across Sections 3.9 and 4.3 into a single reference, since a developer debugging a failing manual test case needs to find the right enum quickly without searching three different header files.

Error Code	Raised By	Meaning
`ERROR_FILE_NOT_FOUND`	Compressor	The input path supplied at the menu prompt does not exist or is not readable.
`ERROR_INPUT_EMPTY`	Compressor	Reserved for future strict-mode use; empty files are a supported edge case (Archive Format Specification §5.1), not an error, in Release 1.0.
`ERROR_CANNOT_WRITE_ARCHIVE`	Compressor / ArchiveWriter	The destination path is not writable (permissions, invalid directory, or disk full).
`ERROR_INVALID_MAGIC`	IntegrityEngine / ArchiveReader	The first four bytes of the file being opened are not the ASCII sequence "JACK".
`ERROR_UNSUPPORTED_VERSION`	IntegrityEngine / ArchiveReader	`format_version` is greater than the highest version this build understands.
`ERROR_CORRUPT_HEADER`	IntegrityEngine	`symbol_count` exceeds 256 or another structural header field is out of range.
`ERROR_TRUNCATED_ARCHIVE`	IntegrityEngine	The physical file is shorter than the header's declared sections imply it should be.
`CHECKSUM_MISMATCH`	IntegrityEngine / Decompressor	The CRC-32 of the restored bytes does not match the checksum stored at compression time.

4.5 UX Design (Interactive CLI Navigation)

```
+------------------------------------------------------------+
|          JACKFRUIT HUFFMAN COMPRESSION TOOL                |
+------------------------------------------------------------+
|  1. Compress File                                          |
|  2. Decompress File                                        |
|  3. Inspect Compression Statistics                         |
|  4. Validate Archive Integrity                             |
|  5. Exit                                                   |
+------------------------------------------------------------+
Choice [1-5]: 1
Enter path to file to compress: test.txt
Output archive path [test.txt.jack]: [ENTER]
[====================] 100% Compressing…
Success! Archive created: test.txt.jack
  - Original Size:  1,024,000 bytes (1.02 MB)
  - Compressed Size: 580,200 bytes (580 KB)
  - Space Saved:    43.34%
  - Ratio:           1.76:1
  - Execution Time:  84.2 ms
```

4.6 Open Issues & Next Steps

- **Sprint 2:** implement `FrequencyTable`, `HuffmanTree`, `BitWriter` (Saatwik), `ArchiveWriter` / `ArchiveReader` (Yuvraj), the standalone `BitReader` against hand-built test bitstreams (Tanmayi), and the CRC-32 engine in isolation (Vibhav).
- **Sprint 3:** integrate the compression vertical slice end-to-end, including the interactive "Compress" CLI screen; wire GitHub Actions coverage reporting.
- **Sprint 4:** round-trip integration (Tanmayi + Saatwik + Yuvraj) — compress → decompress byte-identical on text, binary, empty, and large files; this is the team-wide milestone the Sprint 5 work for UC-03/UC-04 depends on.
- **Sprint 5:** finish the interactive Statistics and Validate screens; wire SonarCloud into the pipeline; write all 40 manual

test cases (not yet executed).

- **Sprint 6:** quality hardening — close SonarCloud critical findings, run ASan/UBSan across the full pipeline, begin the formal security review.
- **Sprint 7:** measure the performance baseline against JACK-NF-001/002 rather than assuming it is met; run the fuzzed-input batch for JACK-NF-003; finalize release packaging.
- **Sprint 8:** execute all 40 manual test cases, complete the Requirement Traceability Matrix, draft the maintenance plan, and assemble the final report.
- **Open issue (not yet resolved):** whether `IS_UNCOMPRESSED` fallback (flag bit 1 in the Archive Format Specification) is implemented in Release 1.0 or deferred — currently the format reserves the bit but no module sets it, which is acceptable for Sprint 1–5 but should be decided explicitly before the Sprint 6 security pass closes out JACK-NF-003 fuzzing, since a pathological already-compressed input could otherwise expand rather than shrink.

5. Appendices

5.1 Glossary

- **BitWriter / BitReader** — utility classes to pack and unpack variable-length bits into and from 8-bit stream bytes, under the shared MSB-first contract.
- **Canonical Huffman Tree** — a Huffman tree standardized by frequency ordering (ties broken by lowest byte value) to ensure two independent implementations produce an identical tree from the same input.
- **CRC-32** — 32-bit Cyclic Redundancy Check (IEEE 802.3 polynomial) used for end-to-end file integrity validation.
- **Prefix Code** — a code system where no code is a prefix of another code, which is what allows a Huffman-encoded bitstream to be decoded unambiguously without delimiters between symbols.
- **Layered Architecture** — the architectural style adopted in Section 3.5, where each layer depends only on the layer directly beneath it.
- **Symbol** — in this specification, a single byte value (0–255) from the input file, treated as the unit of Huffman encoding.
- **Vertical Slice** — a thin but complete implementation of one use case end-to-end (CLI → core logic → archive I/O), used throughout the sprint plan as the unit of incremental delivery.

5.2 References

- IEEE Std 1016-2009: IEEE Standard for Information Technology—Systems Design—Software Design Descriptions.
- ISO/IEC/IEEE 42010: Systems and software engineering — Architecture description.
- D.A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes", 1952.

5.3 Tools Used

- **Language / Compiler:** C++17 (Clang++ 17 / G++ 11).
- **Build System:** CMake 3.28+.
- **Modelling:** Hand-authored SVG diagrams (component, use-case, and sequence models).
- **Testing:** GoogleTest v1.14.0 via CTest.

— End of Software Architecture and Design Specification v0.1.0 —